

ユニット 5・第7週

パイプとプロセスの合成

📅 第7週・日程は未定 📄 演習 TD4（解答 + 生成AIチャレンジ） 📖 アニメーション 2 本

マニュアル M5・ユニットの理論

パイプとプロセスの合成

ユニット 5 の基本の読み物（約1時間）。出力はすべて実物で、コースのマシンで TD4 のプログラムを実行したものです。

目次

- §1 パイプ：2つの端を持つバッファ §4 N 個のコマンドのパイプライン
- §2 規則：ブロック、ファイル終端、SIGPIPE §5 その先へ
- §3 `dup2` でリダイレクトして実行する

§1 パイプ：2つの端を持つバッファ

`pipe(fds)` はカーネル内に循環バッファを作り、2つのディスクリプタを返します。**読み込み端**の `fds[0]` と、**書き込み端**の `fds[1]` です。一方に書いたものが、もう一方から順番どおり、メッセージの区切りのないバイトストリームとして読めます。パイプには名前がありません（ユニット 3 の FIFO とは違います）。使えるのはディスクリプタを受け継いだプロセスだけなので、`fork` の**前**に作ります。その後、各プロセスは**使わない端を閉じます**。書く親は `fds[0]` を、読む子は `fds[1]` を閉じます。

C tds/td4/mypipe.c

```
/*
 * TD4 - exercises 1 and 2: an unnamed pipe between a parent (writer) and a child
 (reader).
 * ./mypipe [count]        the parent writes "Hello World\n" `count` times (default 10)
 * The child reads AT MOST 500 bytes, once, prints them and leaves.
 *
 * Exercise 2: with a large count (10000 x 12 bytes = 120 KB > 64 KiB, the pipe
 capacity)
 * the parent blocks on write() once the pipe is full. When the child exits, nobody can
 read
 * any more: the next write() raises SIGPIPE and kills the parent ("Broken pipe", status
 141
 * in the shell). That only works if the parent CLOSED its own read end: with
 * -DKEEP_READ_END (Makefile target mypipe_bug) the parent itself is still a potential
 reader,
 * the kernel never raises SIGPIPE and the parent blocks forever.
```

```

*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define MESSAGE    "Hello World\n"
#define READ_MAX   500

int main(int argc, char *argv[])
{
    int count = argc > 1 ? atoi(argv[1]) : 10;
    int fds[2];                                /* fds[0] read end, fds[1] write
end */

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                             /* child: reader */
        exit_if(close(fds[1]) == -1, "close"); /* never writes */
        char buffer[READ_MAX];
        ssize_t n = read(fds[0], buffer, READ_MAX); /* one single read */
        exit_if(n == -1, "read");
        exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
        fprintf(stderr, "I am the child, I read %zd bytes and I leave\n", n);
        exit_if(close(fds[0]) == -1, "close");
        return EXIT_SUCCESS;
    }

    /* parent: writer */
#ifdef KEEP_READ_END
    exit_if(close(fds[0]) == -1, "close"); /* THE line that exercise 2 is
about */
#endif
    for (int i = 0; i < count; i++)
        exit_if(write_all(fds[1], MESSAGE, strlen(MESSAGE)) == -1, "write");
    fprintf(stderr, "I am the parent, I wrote %d messages and I leave\n", count);
    exit_if(close(fds[1]) == -1, "close");
    exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
    return EXIT_SUCCESS;
}

```

実際の出力

```
$ ./mypipe
I am the parent, I wrote 10 messages and I leave
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
I am the child, I read 120 bytes and I leave
```

実践 → TD4・1

§2 規則：ブロック、ファイル終端、SIGPIPE

- 空のパイプから読むと、データが届くまでブロックします...ただし、どのプロセスにも書き込み用ディスクリプタが1つも開いていなければ、`read` は 0（ファイル終端）を返します。
- 満杯のパイプに書くと、誰かが読むまでブロックします。Linux の既定の容量は 64 KiB（65,536 バイト）で、`fcntl(F_GETPIPE_SZ / F_SETPIPE_SZ)` で確認・変更できます。
- 読み手が1つも残っていないのに書くと `SIGPIPE` シグナルが発生し、既定ではプロセスが終了します（シェルではステータス 141 = 128 + 13）。無視していれば `write` は `EPIPE` で失敗します。
- `PIPE_BUF` = 4096 バイトまでの書き込みはアトミックで、他の書き手の書き込みと混ざりません。

TD4・2 は最初の 3 つの規則を一度に見せます。子は 500 バイトを 1 回読んで終了し、親は 12 バイトのメッセージを N 個書きます。子が去る前に親が書き終えれば問題ありませんが、子が先に去ると、親の次の `write` が `SIGPIPE` を受けます。メッセージが多ければ必ずそうなりますが、その中間では、別々のコアで動く 2 つのプロセスの競合になり、結果は実行ごとに変わります。サイズごとに 30 回測定した結果：

実際の出力

```
$ for n in 50 100 ... 1000; do run ./mypipe n 30 times; count exit statuses; done
50 messages: 30 x status 0      0 x status 141 (SIGPIPE)
100 messages: 30 x status 0     0 x status 141 (SIGPIPE)
200 messages: 28 x status 0     2 x status 141 (SIGPIPE)
300 messages: 19 x status 0    11 x status 141 (SIGPIPE)
400 messages:  5 x status 0    25 x status 141 (SIGPIPE)
500 messages:  5 x status 0    25 x status 141 (SIGPIPE)
1000 messages: 0 x status 0    30 x status 141 (SIGPIPE)
```

親が `fds[0]` を閉じ忘れると、親自身が読み手として数えられます。 `SIGPIPE` は決して届かず、64 KiB を満たして「誰か」が読むのを永遠に待ち続けます：

実際の出力

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 = killed by
timeout: it was blocked)"
I am the child, I read 500 bytes and I leave
exit status: 124 (124 = killed by timeout: it was blocked)
```

実践 → A5.2 容量と競合・TD4・2

§3 `dup2` でリダイレクトして実行する

`dup2(fds[1], STDOUT_FILENO)` はディスクリプタ1がパイプを指すようにします（ユニット3：オープンファイル表の項目へのポインタをコピーする）。 `exec` はディスクリプタを保つので（ユニット4）、その後に実行したプログラムは知らないうちにパイプに書くことになります。こうして、 `ps` のような他人のプログラムを自分の読み手につなぎます：

C tds/td4/upper.c

```
int main(void)
{
    int fds[2];

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {
        /* child */
        exit_if(close(fds[1]) == -1, "close"); /* otherwise it never sees end of file */
        /*
        to_upper(fds[0], STDOUT_FILENO);
        return EXIT_SUCCESS;
        */

        /* parent */
        exit_if(close(fds[0]) == -1, "close");
        exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
        exit_if(close(fds[1]) == -1, "close"); /* 1 already refers to the pipe */
        execlp("ps", "ps", (char *)NULL); /* "ps" twice: file to run, then
        argv[0] */
        exit_if(1, "execlp ps");
        return EXIT_FAILURE;
    }
}
```

実際の出力

```
$ ./upper | head -4
PID TTY          TIME CMD
6706 ?           00:00:25 UVICORN
9771 ?           00:00:03 SYSTEMD
9801 ?           00:00:00 (SD-PAM)
```

実践 → TD4・3

§4 N 個のコマンドのパイプライン

$c_1 \mid c_2 \mid \dots \mid c_N$ に対して、シェルはパイプを $N-1$ 本、子を N 個作ります。アルゴリズムは、前のパイプの読み込み端を保持する変数 `in` を使った1つのループに収まります。各コマンドについて、最後でなければ `pipe`、次に `fork`。子は `dup2(in, 0)` と `dup2(fds[1], 1)` を行い、それ以外をすべて閉じて `exec` を呼びます。親は `in` と `fds[1]` を閉じ、`fds[0]` を新しい `in` として残します。最後に

全員を待ち、**最後の**コマンドのステータスを返します（bash の `set -o pipefail` は、失敗したうち最後のもののステータスを返します）。

鉄則：どのパイプも、それを使う 2 つのプロセスに、書き込み用と読み込み用のディスクリプタがちょうど 1 つずつ残るようにする。 `close` を 1 つ忘れるだけで（親でも、パイプを受け継いだ子でも）余分な書き手が残り、最後のコマンドは決して来ないファイル終端を待ち続けます。

C tds/td4/pipeline_args.c

```
int in = STDIN_FILENO;
pid_t last = -1;
for (int i = 0; i < ncommands; i++) {
    int fds[2] = { -1, -1 };
    if (i < ncommands - 1)
        exit_if(pipe(fds) == -1, "pipe");

    pid_t pid = fork();
    exit_if(pid == -1, "fork");
    if (pid == 0) {
        if (in != STDIN_FILENO) {
            exit_if(dup2(in, STDIN_FILENO) == -1, "dup2 stdin");
            exit_if(close(in) == -1, "close");
        }
        if (fds[1] != -1) {
            exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2 stdout");
            exit_if(close(fds[0]) == -1 || close(fds[1]) == -1, "close");
        }
        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
    last = pid;
}
```

実際の出力

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

実際の出力

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

実践 → A5.1 パイプラインを組む・TD4・4 と 5

§5 その先へ

- `pipe2(fds, O_CLOEXEC)` は、`exec` 時に閉じる印を付けたディスクリプタでパイプを作ります。起動したプログラムが自分のものではない端を受け継ぐのを防ぎます（§4 のように、それがファイル終端を妨げることがあります）。
- `splice` と `tee` は、パイプとファイルやソケットの間でデータをカーネル内で移し、プロセスの空間にコピーしません。
- `bash` の**プロセス置換**（`diff <(ls a) <(ls b)`）：シェルがパイプを作り、`/dev/fd/N` というパスとして渡します。
- **双方向**の通信にはパイプ 2 本か `socketpair`（ユニット 8）が必要です。